ASP.NET Core / .NET Core.

Interviewer Mindset: For an **8+ years** candidate, I expect not only definitions but also explanations of **why**, **when**, **how**, **real-world usage**, **best practices**, and **common pitfalls**. I'll present the questions exactly as a senior interviewer would.

Module 3 – ASP.NET Core / .NET Core

Part 1 (Questions 1–5)

Question 1: Explain the Architecture of ASP.NET Core.

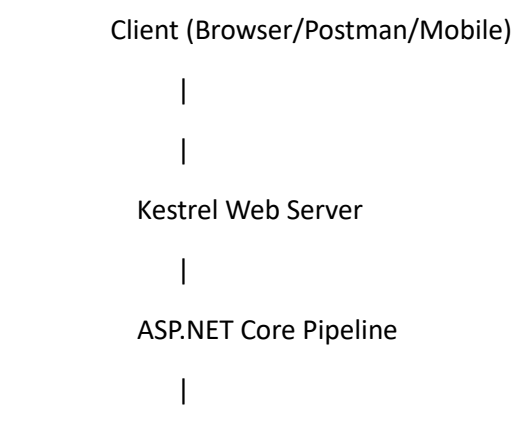
Interviewer

Explain the ASP.NET Core architecture and what happens when a request reaches your application.

Answer

ASP.NET Core is a **cross-platform, high-performance, open-source web framework** used to build Web APIs, MVC applications, Minimal APIs, SignalR applications, and gRPC services.

High-Level Architecture



|

Controller / Minimal API

|

Business Layer

|

Repository Layer

|

Entity Framework Core

|

SQL Server / PostgreSQL

Request Flow

Suppose the client calls:

GET /api/employees/100

The flow is:

Client

↓

Kestrel

↓

Middleware Pipeline

↓

Routing

↓

Authentication

↓

Authorization

↓

Controller

↓

Service Layer

↓

Repository

↓

Database

The response travels back through the middleware pipeline to the client.

Interview Follow-up

Why is ASP.NET Core faster than ASP.NET MVC (.NET Framework)?

Expected answer:

- Lightweight pipeline
- Cross-platform
- Built on Kestrel
- Optimized dependency injection
- Asynchronous by default
- High-performance JSON serialization (System.Text.Json by default)

Real-Time Example

Suppose you're building an Insurance Claims API:

Customer

↓

POST /api/claims

Authentication

↓

ClaimController

↓

ClaimService

↓

ClaimRepository

↓

SQL Server

Each layer has a single responsibility.

Best Practices

✓ Keep Controllers thin.

✓ Business logic belongs in the Service layer.

✓ Database code belongs in the Repository layer.

✓ Use Dependency Injection.

✓ Separate concerns.

Question 2: Explain Kestrel Web Server.

Very commonly asked.

Answer

Kestrel is the **cross-platform web server** used by ASP.NET Core.

It handles:

- HTTP
- HTTPS
- HTTP/2
- HTTP/3 (supported in modern ASP.NET Core versions)

Architecture

Internet

↓

Kestrel

↓

Middleware

↓

Controllers

Why Kestrel?

Advantages

- Very fast
- Lightweight
- Asynchronous
- Cross-platform
- High throughput

Interview Question

Does Kestrel replace IIS?

Answer

No.

In production on Windows, IIS often acts as a **reverse proxy**.

Internet

↓

IIS

↓

Kestrel

↓

Application

On Linux

Nginx

↓

Kestrel

↓

Application

Real Interview Question

Why not expose Kestrel directly to the Internet?

Expected answer

Reverse proxies provide:

- SSL termination
- Load balancing
- Request filtering
- Compression
- Better security
- Logging
- Web Application Firewall (depending on infrastructure)

Question 3

Explain Middleware

This is one of the most frequently asked ASP.NET Core questions.

Answer

Middleware is software that processes HTTP requests and responses.

Every request passes through a pipeline of middleware components.

Diagram

Request

↓

Middleware 1

↓

Middleware 2

↓

Middleware 3

↓

Controller

↓

Response

Built-in Middleware

- Exception Handler
- HTTPS Redirection
- Static Files
- Routing
- Authentication
- Authorization
- CORS
- Response Compression

Custom Middleware Example

```
public class RequestLoggingMiddleware
{
   private readonly RequestDelegate _next;

   public RequestLoggingMiddleware(RequestDelegate next)
   {
      _next = next;
   }

   public async Task Invoke(HttpContext context)
   {
      Console.WriteLine($"Request: {context.Request.Path}");

      await _next(context);

      Console.WriteLine($"Response: {context.Response.StatusCode}");
   }
}
```

Register it:

```
app.UseMiddleware<RequestLoggingMiddleware>();
```

Interview Question

Difference between:

app.Use()

app.Run()

app.Map()

Use()

Calls the next middleware.

Middleware A

↓

Middleware B

↓

Middleware C

Run()

Terminal middleware.

Stops the pipeline.

Map()

Branches the pipeline based on URL path.

Example

/api

↓

Pipeline A

/admin

↓

Pipeline B

Best Practice

The middleware order is critical.

Correct order (typical example):

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

Note: Middleware order depends on the application's needs. For example, UseCors() typically appears between UseRouting() and authentication/authorization in many APIs.

Question 4

Explain Dependency Injection (DI)

This is probably the **No. 1 ASP.NET Core interview question**.

Answer

Dependency Injection is a design pattern where required dependencies are supplied from outside instead of being created inside a class.

Without DI

```
public class EmployeeController
{
    private EmployeeService service = new EmployeeService();
}
```

Problems

- Tight coupling
- Hard to unit test
- Difficult to replace implementations

With DI

```
public class EmployeeController
{
   private readonly IEmployeeService service;

   public EmployeeController(IEmployeeService service)
   {
      this.service = service;
   }
}
```

Registration

```
builder.Services.AddScoped<IEmployeeService, EmployeeService>();
```

Benefits

- Loose coupling
- Easier testing
- Better maintainability
- Easier mocking
- Better architecture

Real-world Example

Controller

↓

IEmployeeService

↓

EmployeeService

↓

IRepository

↓

Repository

↓

SQL Server

Interview Question

Why is constructor injection preferred?

Answer

- Required dependencies are explicit.
- Immutable after construction.
- Easier to test.
- Preferred by the ASP.NET Core framework.

Question 5

Explain Service Lifetimes

One of the most important interview questions.

ASP.NET Core supports three lifetimes.

Singleton

Created once.

Entire Application

↓

One Instance

Registration

builder.Services.AddSingleton<ICacheService, CacheService>();

Use for

- Cache
- Configuration
- Logging
- Shared read-only services

Scoped

One instance per HTTP request.

Request 1

↓

One Instance

Request 2

↓

New Instance

Registration

builder.Services.AddScoped<IEmployeeService, EmployeeService>();

Use for

- Business services
- Repositories
- Entity Framework DbContext

Transient

A new instance every time it's requested.

builder.Services.AddTransient<IEmailService, EmailService>();

Use for

- Lightweight services
- Stateless helpers

Interview Question

Why is DbContext Scoped?

Answer

Because one DbContext instance should represent a single unit of work for a request. Sharing it across requests (Singleton) can cause concurrency issues, while creating too many instances unnecessarily (Transient) can make managing a unit of work more difficult.

Common Interview Questions

Can a Singleton consume a Scoped service?

Answer:
Not directly through constructor injection. A Singleton lives for the lifetime of the application, while a Scoped service is created per request. Injecting a Scoped service into a Singleton creates a lifetime mismatch. If necessary, resolve the Scoped service within an explicitly created scope using IServiceScopeFactory.

Which lifetime is most commonly used?

For business applications:

- **Scoped** → Business services and repositories
- **Singleton** → Configuration, caching, logging abstractions
- **Transient** → Stateless utility services

Interview Tip

A strong senior answer includes not just the definitions but also the reasoning behind the choice of lifetime and the trade-offs involved.

Module 3 Progress

We've covered:

1. ASP.NET Core Architecture
2. Kestrel Web Server
3. Middleware
4. Dependency Injection
5. Service Lifetimes

Next (Part 2)

We'll continue with another set of high-value interview topics:

6. **Configuration & Options Pattern**
7. **appsettings.json and Secret Management**
8. **Logging (ILogger)**
9. **Model Binding**
10. **Model Validation**
11. **Filters (Authorization, Action, Exception, Resource, Result)**
12. **Routing (Conventional vs Attribute Routing)**

These topics are asked in almost every senior ASP.NET Core interview and often include scenario-based follow-up questions.

Excellent. This is **Part 2 of Module 3 (.NET Core / ASP.NET Core)**.

These topics are asked in almost every **8–15 years** .NET interview because they evaluate your understanding of **configuration, logging, request processing, and enterprise application development**.

Module 3 – ASP.NET Core

Part 2

Question 6: Explain Configuration in ASP.NET Core

Interviewer

How does ASP.NET Core load configuration? What are the different configuration providers?

Answer

ASP.NET Core has a **Configuration System** that loads settings from multiple sources.

Configuration is available through

IConfiguration

Configuration Providers

ASP.NET Core loads configuration in the following order (later providers can override earlier ones):

appsettings.json

↓

appsettings.{Environment}.json

↓

User Secrets (Development)

↓

Environment Variables

↓

Command Line Arguments

Example

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=.;Database=EmployeeDB;"
  }
}
```

Reading it

```
public class EmployeeService
```

```
{
    private readonly IConfiguration configuration;

    public EmployeeService(IConfiguration configuration)
    {
        this.configuration = configuration;
    }

    public void Print()
    {
        var conn = configuration.GetConnectionString("DefaultConnection");
    }
}
```

Real-Time Example

Insurance Application

Development

↓

SQL Local

QA

↓

Azure SQL

Production

↓

Managed SQL Server

Different environments use different configuration values.

Interview Question

Can we change configuration without recompiling the application?

Answer

Yes.

Update

appsettings.json

or

Environment Variables

without changing code.

Question 7

Explain Options Pattern

One of Microsoft's recommended approaches.

Answer

Instead of reading configuration everywhere,

Bind configuration to strongly typed classes.

Example

appsettings.json

```
{
  "EmailSettings": {
    "Host": "smtp.gmail.com",
```

```
  "Port": 587
 }
}
```

Class

```
public class EmailSettings
{
    public string Host { get; set; }

    public int Port { get; set; }
}
```

Registration

```
builder.Services.Configure<EmailSettings>(
    builder.Configuration.GetSection("EmailSettings"));
```

Usage

```
public class EmailService
{
    private readonly EmailSettings settings;

    public EmailService(IOptions<EmailSettings> options)
    {
        settings = options.Value;
    }
}
```

Advantages

✓ Strongly Typed

✓ Easy Testing

✓ Clean Code

✓ IntelliSense

Interview Question

Difference between

IConfiguration

and

IOptions

IConfiguration	**IOptions**
Reads raw configuration	Strongly typed
Manual key access	Automatic binding
More flexible	Cleaner for application settings

Interview Tip

For enterprise applications,

Prefer

IOptions<T>

over

IConfiguration

for application settings.

Question 8

Explain Logging in ASP.NET Core

Very common.

Answer

ASP.NET Core has built-in logging using

ILogger<T>

Example

```
public class EmployeeService
{
    private readonly ILogger<EmployeeService> logger;

    public EmployeeService(
        ILogger<EmployeeService> logger)
    {
        this.logger = logger;
    }

    public void Save()
    {
        logger.LogInformation("Employee Saved");
    }
}
```

Log Levels

Level	Purpose
Trace	Detailed diagnostic information
Debug	Development debugging
Information	Normal application flow
Warning	Unexpected but recoverable situations
Error	Operation failed
Critical	Application/service failure

Real Production Example

```
logger.LogInformation(
    "Employee {EmployeeId} updated successfully.",
    employeeId);
```

Use **structured logging** rather than string concatenation.

Interview Question

Why not use

Console.WriteLine()

Answer

Because

- Not structured
- Hard to search
- No log levels
- No centralized storage
- Not production ready

Production Logging

Usually integrated with

- Azure Application Insights
- Serilog
- NLog
- ELK Stack

Question 9

Explain Model Binding

Answer

Model Binding automatically converts HTTP request data into C# objects.

Example Request

POST /api/employees

Body

```
{
  "id":1,
```

"name":"Satheesh"

}

Controller

[HttpPost]

public IActionResult Save(Employee employee)

{

return Ok();

}

ASP.NET Core automatically binds JSON to the Employee object.

Sources

Model Binding can bind from

- Route
- Query String
- Body
- Form
- Headers

Examples

[FromBody]

[FromRoute]

[FromQuery]

[FromHeader]

[FromForm]

Interview Question

What if

[FromBody]

is missing?

Answer

For controllers marked with [ApiController], complex types are typically inferred from the request body automatically. For simple types, the default binding source is different (such as route or query string), unless specified.

Question 10

Explain Model Validation

Very frequently asked.

Answer

ASP.NET Core validates models using Data Annotations.

Example

```
public class Employee
{
   [Required]
   public string Name { get; set; }

   [Range(18,60)]
   public int Age { get; set; }
}
```

Controller

```
if(!ModelState.IsValid)
{
   return BadRequest(ModelState);
}
```

Client

```
{
```

"Age":10

}

Response

Name is required

Age must be between 18 and 60

Common Validation Attributes

Required

StringLength

Range

EmailAddress

Phone

Compare

RegularExpression

Interview Question

Does

[ApiController]

perform validation automatically?

Answer

Yes.

When validation fails,

ASP.NET Core automatically returns

400 Bad Request

with validation errors unless customized.

Question 11

Explain Filters

One of the favorite interview questions.

Types

Filter	Purpose
Authorization	Security
Resource	Before/After resource execution
Action	Before/After action method
Exception	Global exception handling
Result	Before/After response generation

Action Filter

Request

↓

Action Filter

↓

Controller

↓

Action Filter

↓

Response

Real-Time Example

Audit Logging

User Login

↓

Action Filter

↓

Store Audit Log

↓

Controller

Exception Filter

Instead of

try

catch

try

catch

everywhere,

Use

Global Exception Filter

Interview Question

Difference between Middleware and Filters?

Middleware	**Filters**
Entire pipeline	MVC/Controller pipeline
Runs before routing is completed? (depends on placement)	Runs after routing, around controller actions
Handles all requests	Applies to MVC actions/endpoints where appropriate

---

Interview Tip

Use Middleware for

- Logging
- Authentication
- CORS
- Compression

Use Filters for

- Validation
- Audit
- Exception Handling (MVC-specific)
- Action-specific behavior

Question 12

Explain Routing

Answer

Routing maps incoming URLs to endpoints.

Conventional Routing

```
app.MapControllerRoute(
    name:"default",
```

pattern:"{controller=Home}/{action=Index}/{id?}");

Attribute Routing

[Route("api/employees")]

public class EmployeeController : Controller

{

}

Action

[HttpGet("{id}")]

public IActionResult Get(int id)

{

}

Request

GET

/api/employees/10

Which is Preferred?

Enterprise APIs use

Attribute Routing

because

- More readable
- Better versioning
- Better REST support
- Easier maintenance

Interview Question

Difference between

[Route]

[HttpGet]

[HttpPost]

Answer

[Route]

Defines route template.

[HttpGet]

Handles GET request.

[HttpPost]

Handles POST request.

Scenario-Based Question ⭐

Interviewer

A client reports that requests to your API are returning **404 Not Found** even though the controller exists. How would you troubleshoot?

Strong Answer

I would verify the following in order:

1. Check that the endpoint is mapped correctly (e.g., app.MapControllers() is configured).
2. Verify the route attributes and URL template.
3. Confirm the correct HTTP verb (GET, POST, etc.) is being used.
4. Ensure the controller is discovered by the application (correct assembly and namespace if applicable).
5. Review middleware ordering (routing and endpoint mapping).
6. Test the endpoint using Swagger or Postman.
7. Review application logs for routing or startup errors.

Module 3 Progress

We have now completed:

✅ ASP.NET Core Architecture

✅ Kestrel

✅ Middleware

✅ Dependency Injection

✅ Service Lifetimes

✅ Configuration

✅ Options Pattern

✅ Logging

✅ Model Binding

✅ Model Validation

✅ Filters

✅ Routing

Next Part (Part 3)

We'll cover another set of essential senior interview topics:

- Authentication vs Authorization
- JWT Authentication
- ASP.NET Core Identity
- CORS
- Session vs Cookies
- Caching (In-Memory & Distributed)
- Response Caching
- Health Checks
- Background Services (IHostedService)
- Global Exception Handling
- ProblemDetails
- Performance Optimization
- Interview scenarios and production troubleshooting

This next section contains some of the **most frequently asked questions** in senior .NET interviews, especially for roles involving enterprise APIs and Azure-hosted applications.

Excellent. This is **Part 3 of Module 3**, and these are among the **highest-priority interview topics** for **8+ years of experience**. In my experience as a senior interviewer, these questions are asked in almost every enterprise .NET interview because they directly relate to building secure, scalable APIs.

Module 3 – ASP.NET Core

Part 3

Question 13: Explain Authentication vs Authorization

Interviewer

What is the difference between Authentication and Authorization?

This is one of the most commonly asked questions.

Answer

Although the terms are related, they solve different problems.

Authentication	**Authorization**
Verifies **who you are**	Determines **what you are allowed to do**
Happens first	Happens after authentication
Validates identity	Validates permissions
Example: Login	Example: Access control

Real-Time Example

Imagine an Insurance Portal.

Step 1 – Authentication

User enters:

Username : satheesh

Password : ********

System validates credentials.

If correct:

Identity Verified

Step 2 – Authorization

User requests:

DELETE /api/policies/100

System checks role.

If

Admin

Allow.

If

Customer

Return

403 Forbidden

Interview Question

Can Authorization happen without Authentication?

Answer

No.

The application must know **who the user is** before it can determine **what the user is allowed to do**.

Question 14

Explain JWT Authentication

This is probably the **most frequently asked ASP.NET Core interview question**.

Answer

JWT stands for

JSON Web Token

It is a compact, self-contained token used for stateless authentication.

JWT Structure

A JWT has three parts.

Header

.

Payload

.

Signature

Example

xxxxx.yyyyy.zzzzz

Header

Contains metadata.

Example

{

"alg":"HS256",

"typ":"JWT"

}

Payload

Contains claims.

Example

{

"sub":"Satheesh",

"role":"Admin",

"exp":123456789

}

Signature

Generated using a secret key to ensure integrity.

Authentication Flow

Client

|

POST /login

|

Validate Username & Password

|

Generate JWT

|

Return Token

|

Future Requests

Authorization: Bearer <token>

|

JWT Validation

|

Controller

ASP.NET Core Registration

```
builder.Services
.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(options =>
{
    options.TokenValidationParameters =
        new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true
```

```
    };
});
```

Protect API

```
[Authorize]
public class EmployeeController : ControllerBase
{
}
```

Allow Anonymous

```
[AllowAnonymous]
public IActionResult Login()
{
}
```

Interview Question

Why is JWT preferred?

Advantages

✓ Stateless

✓ No server session required

✓ Scalable

✓ Suitable for APIs

✓ Mobile friendly

Follow-up Question

Where should JWT be stored?

Best practice

- **Browser applications:** Prefer **Secure, HttpOnly cookies** for better protection against XSS (depending on your architecture and CSRF strategy).
- **Native/mobile apps:** Use secure platform storage (e.g., Keychain on iOS, Keystore on Android).

Avoid storing long-lived tokens in places that are easily accessible to JavaScript unless you've carefully considered the security trade-offs.

Question 15

Explain ASP.NET Core Identity

Answer

ASP.NET Core Identity is Microsoft's membership system.

It provides:

- User Management
- Password Hashing
- Roles
- Claims
- Two-Factor Authentication
- Email Confirmation
- Password Reset
- Account Lockout

Architecture

User

↓

Identity

↓

SQL Server

↓

AspNetUsers

AspNetRoles

AspNetUserRoles

Interview Question

Difference between

JWT

and

Identity

Answer

Identity

- User Management Framework

JWT

- Authentication Token

They often work together.

Question 16

Explain CORS

Very common.

Answer

CORS

stands for

Cross-Origin Resource Sharing

Browsers block requests from different origins unless permitted.

Example

Frontend

https://company.com

Backend

https://api.company.com

Different origins.

Without CORS

Browser blocks request.

Configuration

```
builder.Services.AddCors(options =>
{
   options.AddPolicy("AllowUI", policy =>
   {
      policy
         .WithOrigins("https://company.com")
         .AllowAnyHeader()
         .AllowAnyMethod();
   });
});
```

Use

```
app.UseCors("AllowUI");
```

Interview Question

Should we use

AllowAnyOrigin()

in Production?

Answer

No.

Restrict to trusted domains whenever possible.

Question 17

Explain Session vs Cookies

Cookies

Stored

Client Side

Used for

- Preferences
- Theme
- Authentication (often via secure cookies)

Session

Server-managed state associated with a client.

Used for

- Temporary user-specific data
- Shopping carts (traditional web apps)
- Multi-step workflows

Interview Question

Which one is used in Web APIs?

Answer

Modern REST APIs are generally **stateless**, so they typically use **JWT or OAuth-based authentication** rather than server-side session state.

Question 18

Explain Caching

Very important.

Caching improves

- Performance
- Response Time
- Scalability

Types

In-Memory Cache

builder.Services.AddMemoryCache();

Stores data in the application's memory.

Suitable for

Single Server

Distributed Cache

Examples

- Redis
- SQL Server

Suitable for

Load Balancer

↓

Server 1

↓

Redis

↓

Server 2

All servers share the same cache.

Interview Question

Which cache should be used in Azure?

Answer

Azure Cache for Redis.

Best Practices

Do not cache

- Highly sensitive information
- Frequently changing data without a proper invalidation strategy

---

Question 19

Explain Health Checks

Health Checks monitor application health.

Example

builder.Services.AddHealthChecks();

Endpoint

app.MapHealthChecks("/health");

Response

Healthy

or

Unhealthy

Azure Example

Azure App Service

↓

Calls

/health

If unhealthy,

Application can be restarted or taken out of rotation depending on the hosting configuration.

Question 20

Explain Background Services

ASP.NET Core supports background processing using

BackgroundService

or

IHostedService

Example

```
public class EmailService : BackgroundService
{
   protected override async Task ExecuteAsync(
      CancellationToken stoppingToken)
   {
      while (!stoppingToken.IsCancellationRequested)
      {
         await SendEmailsAsync();

         await Task.Delay(5000, stoppingToken);
      }
   }
}
```

Registration

```
builder.Services.AddHostedService<EmailService>();
```

Real-Time Example

Every minute

↓

Check Queue

↓

Send Email

↓

Update Database

Interview Question

Should long-running work be done inside Controllers?

Answer

No.

Controllers should return responses quickly.

Long-running operations should be delegated to background services, queues, or messaging systems (e.g., Azure Service Bus, Hangfire, Quartz.NET, depending on the architecture).

⭐ **Scenario-Based Question**

Interviewer

A Web API becomes slow after deployment.

How would you investigate?

Strong Answer

I would take a structured approach:

1. Check application logs (ILogger) and telemetry (e.g., Application Insights).
2. Measure API response times and identify the slow endpoints.
3. Review SQL queries and execution plans for bottlenecks.
4. Check CPU, memory, thread pool usage, and connection pool utilization.
5. Verify whether external dependencies (REST APIs, storage, queues) are responding slowly.
6. Review caching opportunities and cache hit rates.
7. Confirm that async APIs are used correctly and avoid blocking calls like .Result or .Wait().
8. Load test after applying fixes to validate improvements.

This kind of systematic troubleshooting demonstrates senior-level production support experience.

Module 3 Progress

We've now covered:

- ASP.NET Core Architecture
- Kestrel
- Middleware
- Dependency Injection
- Service Lifetimes

- Configuration
- Options Pattern
- Logging
- Model Binding
- Model Validation
- Filters
- Routing
- Authentication
- Authorization
- JWT
- ASP.NET Core Identity
- CORS
- Session vs Cookies
- Caching
- Health Checks
- Background Services

Next Part (Part 4 – Final)

We'll complete Module 3 with advanced enterprise topics:

- Global Exception Handling
- Exception Middleware
- ProblemDetails (RFC 7807)
- Response Compression
- Response Caching
- Output Caching (.NET 8+)
- Rate Limiting
- Minimal APIs
- SignalR
- gRPC
- API Versioning
- Performance Optimization

- Security Best Practices
- Top 50 Rapid-Fire ASP.NET Core Interview Questions
- Real Production Scenarios (the kind asked by Microsoft, Deloitte, EY, Accenture, Cognizant, and Capgemini)

These are the topics that typically distinguish a senior developer from a mid-level developer in interviews.

Excellent. You're now entering the **expert-level ASP.NET Core topics**. These are the kinds of questions I ask when interviewing **Senior .NET Developers (8–15 years)** for enterprise applications.

This section focuses on **production-ready API development**, **performance**, **security**, and **real-world troubleshooting**.

Module 3 – ASP.NET Core

Part 4 (Expert Level)

Question 21: How do you implement Global Exception Handling?

Interviewer

How do you handle exceptions globally in ASP.NET Core?

Answer

Instead of placing **try-catch** blocks in every controller, ASP.NET Core supports centralized exception handling through **Exception Handling Middleware**.

❌ **Bad Practice**

```
[HttpGet]
public IActionResult Get()
{
   try
   {
      var employees = service.GetEmployees();
      return Ok(employees);
   }
   catch(Exception ex)
   {
```

```
        return StatusCode(500);
    }
}
```

Problems

- Duplicate code
- Difficult to maintain
- Inconsistent responses

✅ **Good Practice**

Create Exception Middleware.

```
public class ExceptionMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(RequestDelegate next,
                               ILogger<ExceptionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task Invoke(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch(Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception");
```

```
        context.Response.StatusCode = 500;


        await context.Response.WriteAsJsonAsync(
            new
            {
                Message="Unexpected Error"
            });
      }
   }
}
```

Register

```
app.UseMiddleware<ExceptionMiddleware>();
```

Interview Question

Why Middleware instead of Exception Filter?

Answer

Middleware	**Exception Filter**
Handles entire HTTP pipeline	MVC/Controller actions only
Captures exceptions before MVC	MVC-specific
Recommended for Web APIs	Useful for MVC-specific behavior

Interview Tip

Senior developers generally prefer centralized exception handling using middleware because it provides a consistent error response for the entire application.

Question 22

What is ProblemDetails?

Very common in .NET 8 interviews.

Answer

ProblemDetails is the standard format for API error responses defined by **RFC 7807**.

Instead of returning

```
{
  "message":"Error"
}
```

Return

```
{
  "type":"https://example.com/errors/validation",
  "title":"Validation Failed",
  "status":400,
  "detail":"Name is required",
  "instance":"/api/employees"
}
```

Advantages

✓ Standardized

✓ Better API documentation

✓ Easier client integration

✓ Recommended by Microsoft

.NET 8 Registration

```
builder.Services.AddProblemDetails();
```

Interview Question

Why use ProblemDetails?

Answer

It provides a consistent, machine-readable error format that clients can parse reliably.

Question 23

Explain Response Compression

Large JSON responses

↓

Consume bandwidth

↓

Slow API

Enable compression

builder.Services.AddResponseCompression();

Use

app.UseResponseCompression();

Client

↓

100 KB

↓

Compression

↓

20 KB

Benefits

✓ Faster API

✓ Less bandwidth

✓ Better performance

Question 24

Response Caching vs Output Caching

Very common.

Response Caching

Uses HTTP cache headers.

Works only when

Client

or

Proxy

supports caching.

Example

[ResponseCache(Duration=60)]

Output Caching (.NET 8)

Stores response inside ASP.NET Core.

Registration

builder.Services.AddOutputCache();

Use

app.UseOutputCache();

Controller

[OutputCache(Duration=60)]

Interview Question

Which is better?

Answer

For server-side performance in modern ASP.NET Core applications, **Output Caching** is generally more powerful and flexible. **Response Caching** relies on HTTP caching semantics and intermediary/client behavior.

Question 25

Explain Rate Limiting

Very popular in .NET 8.

Suppose

10000 Requests

↓

One API

↓

Server Crash

Solution

Rate Limiting

Registration

builder.Services.AddRateLimiter(options =>

{

});

Policy Example

100 Requests

Per Minute

Benefits

✓ Prevent API abuse

✓ Prevent DoS attacks

✓ Improve scalability

Interview Question

Where is Rate Limiting useful?

- Login API
- Payment API
- OTP API
- Public APIs

Question 26

Explain Minimal APIs

Introduced in .NET 6.

Traditional

Controller

↓

Action

↓

Route

Minimal API

```
app.MapGet("/employees", () =>
{
   return employees;
});
```

Advantages

✓ Less code

✓ Faster startup

✓ Small APIs

Interview Question

Should Enterprise Applications use Minimal APIs?

Answer

It depends.

Small APIs

✓ Yes

Large Enterprise

Controllers remain more suitable for many teams because they provide richer features, organization, filters, model binding conventions, and easier maintainability.

Question 27

Explain SignalR

SignalR provides

Real-Time Communication

Example

Chat Application

User A

↓

Server

↓

User B

Immediately

Other Examples

- Live Stock Market
- Cricket Score
- Notifications
- Dashboard
- Chat
- GPS Tracking

Interview Question

How does SignalR communicate?

Uses best available transport.

Typically:

- WebSockets (preferred)
- Server-Sent Events
- Long Polling

Question 28

Explain gRPC

---

gRPC

↓

High Performance RPC Framework

Uses

Protocol Buffers

instead of JSON.

Comparison

REST	**gRPC**
JSON	Protocol Buffers
Larger payload	Smaller payload
Human-readable	Binary
Good for public APIs	Excellent for service-to-service communication

---

Interview Question

Should Mobile Apps use gRPC?

Answer

Depends on requirements.

REST remains common for public/mobile APIs due to broad ecosystem support, while gRPC is excellent for high-performance internal microservice communication. Modern mobile clients can also use gRPC when appropriate.

Question 29

Explain API Versioning

Example

/api/v1/employees

/api/v2/employees

Methods

URL

Header

Query String

Media Type

Interview Tip

Most enterprise APIs prefer

URL Versioning

because it is simple and explicit, although organizations choose different strategies based on their API governance.

Question 30

Performance Optimization in ASP.NET Core

Very frequently asked.

Ways to Improve Performance

✓ Async Programming

✓ Response Compression

✓ Output Caching

✓ Redis Cache

✓ Connection Pooling

✓ Pagination

✓ Database Indexing

✓ HTTP/2 or HTTP/3 where appropriate

✓ Avoid unnecessary object creation

✓ Structured Logging

✓ Minimize synchronous blocking

Production Scenario 1 ⭐

Interviewer

Your API response time increased from **500 ms** to **5 seconds** after deployment.

How do you investigate?

Strong Answer

I would investigate in the following order:

Step 1

Application Insights

↓

Request Duration

↓

Dependency Duration

↓

Exceptions

Step 2

SQL

↓

Execution Plan

↓

Indexes

↓

Blocking

↓

Deadlocks

Step 3

External APIs

↓

Latency

↓

Retries

↓

Timeouts

Step 4

Memory

↓

CPU

↓

GC

↓

ThreadPool

Step 5

Load Test

↓

Verify improvements

Production Scenario 2 ⭐

Interviewer

Users receive intermittent **500 Internal Server Error** responses.

How do you troubleshoot?

Expected Senior Answer

- Review application logs with correlation IDs.
- Check Application Insights or centralized logging.
- Identify the failing endpoint and exception type.
- Verify database connectivity and connection pool health.
- Check downstream services (REST APIs, Service Bus, Storage, etc.).
- Review recent deployments and configuration changes.
- Validate environment variables and secrets.
- Reproduce the issue in a lower environment if possible.
- Apply a fix, deploy, and monitor telemetry to confirm resolution.

Top 15 Rapid-Fire ASP.NET Core Questions

Q1. Difference between Use() and Run()?

Answer: Use() passes control to the next middleware. Run() is terminal middleware and ends the pipeline.

Q2. Why is DbContext Scoped?

Answer: One instance per request aligns with the unit-of-work pattern and avoids concurrency issues.

Q3. What is Dependency Injection?

Answer: A design pattern that supplies dependencies from the outside, reducing coupling and improving testability.

Q4. Why should controllers be thin?

Answer: Controllers should handle HTTP concerns only. Business logic belongs in services.

Q5. What is Middleware?

Answer: Software components that process HTTP requests and responses in a pipeline.

Q6. What is Model Binding?

Answer: Automatically maps request data to C# objects.

Q7. What is Model Validation?

Answer: Validates incoming models using data annotations or custom validation.

Q8. Difference between Authentication and Authorization?

Answer: Authentication verifies identity. Authorization verifies permissions.

Q9. Why use JWT?

Answer: Stateless authentication that scales well for APIs.

Q10. Why use Redis?

Answer: Distributed caching shared across multiple application instances.

Q11. What is CORS?

Answer: A browser security mechanism that controls cross-origin requests.

Q12. What is Kestrel?

Answer: ASP.NET Core's cross-platform web server.

Q13. What is IHostedService?

Answer: A framework abstraction for running background tasks.

Q14. What is ProblemDetails?

Answer: A standardized API error response format based on RFC 7807.

Q15. What is Output Caching?

Answer: A server-side caching feature that stores responses to improve performance.

🎯 Senior Interview Advice

For **8+ years**, interviewers are looking beyond syntax. They want evidence that you can:

- Design clean, maintainable architectures.
- Build secure and scalable APIs.
- Troubleshoot production issues methodically.
- Explain trade-offs between different approaches.
- Mentor junior developers and justify technical decisions.

A strong answer combines:

1. **Concept**
2. **Real-world example**
3. **Best practice**
4. **Common pitfalls**
5. **Production experience**

Module 3 Status

You now have comprehensive coverage of **ASP.NET Core / .NET Core**, including:

- Architecture
- Middleware
- Dependency Injection
- Configuration
- Logging
- Routing
- Validation
- Authentication & JWT
- Identity
- CORS
- Caching
- Health Checks
- Background Services
- Exception Handling
- ProblemDetails

- Output Caching
- Rate Limiting
- Minimal APIs
- SignalR
- gRPC
- API Versioning
- Performance Optimization
- Real production scenarios
- Rapid-fire interview questions

This level of preparation is aligned with what senior interviewers expect from experienced .NET developers in enterprise environments.